University ERP System

Architecture and Design Documentation

Hardik
Roll No: 2024227

Harsh
Roll No: 2024234

Course: Advanced Programming (CSE201)
Institute: IIITD

1 Use Case Overview

The ERP system relies on a strict layered architecture: **UI** → **Service** → **DAO** → **Database**. The system is divided into three primary roles.

1.1 Student Use Cases

Use Case	Implementation Detail
Login	Authenticates via `AuthService` against the `users_auth` table.
Browse Catalog	Displays Course/Section details and live seat availability via `CatalogService`.
Register	`StudentService.registerForSection`: Checks Maintenance, Deadline, Duplicates, and Capacity.
Drop Section	`StudentService.dropSection`: Updates status to 'DROPPED' if allowed.
View Grades	`StudentService.getGrades`: Fetches final and component grades.
Download Transcript	Exports completed courses to CSV using `TranscriptCsvExporter`.

1.2 Instructor Use Cases

Use Case	Implementation Detail
View Sections	Lists assigned sections via `InstructorService.getMySections`.
Configure Schema	Saves custom assessment weights (e.g., Quiz=40%) to `assessment_schema`.
Enter Grades	Saves/Updates scores via `GradeDAO`.
Compute Grades	Calculates weighted averages and assigns letter grades based on logic in `InstructorService`.
View Statistics	Displays Class Average, Highest, and Lowest scores.

1.3 Admin Use Cases

Use Case	Implementation Detail
Manage Users	Creates Student/Instructor accounts in both `users_auth` and profile tables.
Manage Courses	CRUD operations for Master Course list.
Manage Sections	Schedules sections (Time/Room) and assigns Instructors.
Maintenance Mode	Toggles global read-only mode via `AdminService`.

2 Domain Model

This diagram represents the Java POJO classes (‘edu.univ.erp.domain‘) and the underlying database schema. It highlights the "Shared ID" strategy used to link Authentication data with User Profiles.

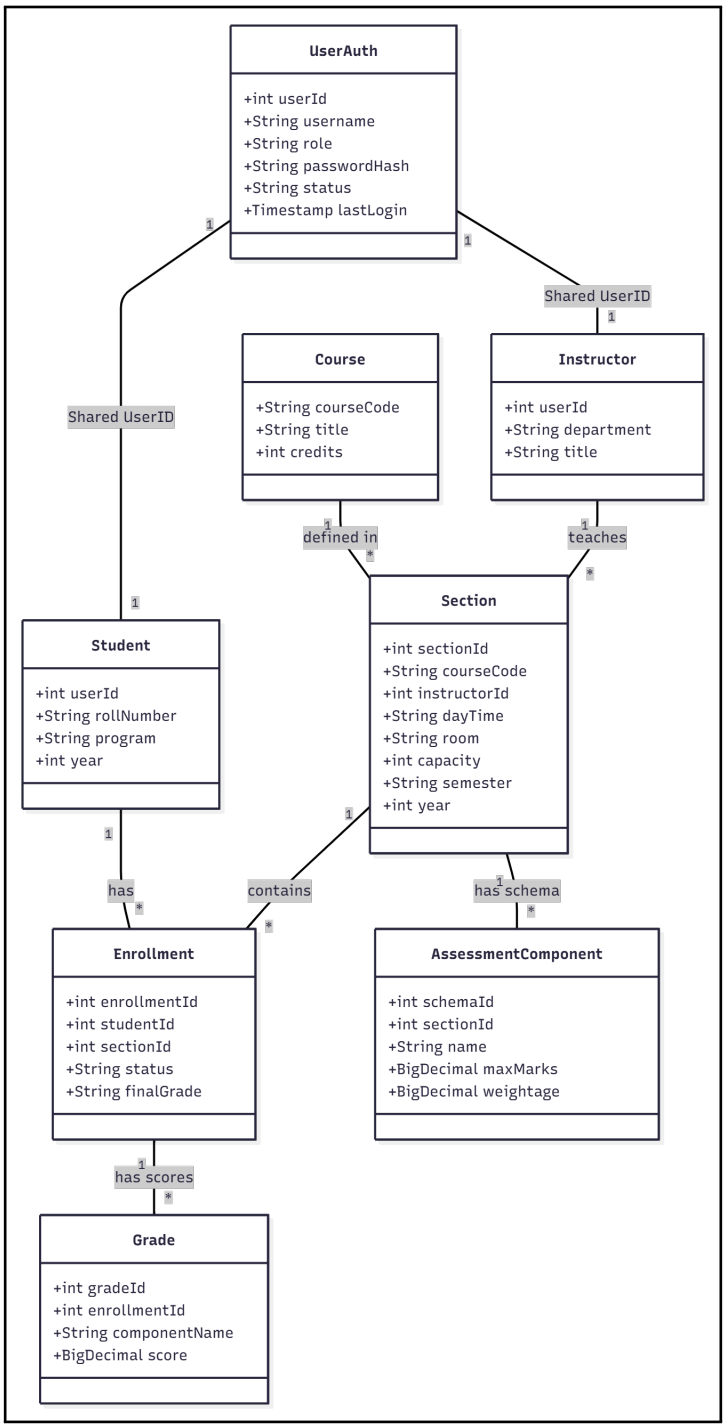


Figure 1: Domain Model (Entities and Relationships)

3 Flow Diagrams

3.1 Student Enrollment Flow

This flow represents the logic inside `StudentService.registerForSection`. It enforces a strict order of operations: System Check → Deadline Check → Data Validation → Database Write.

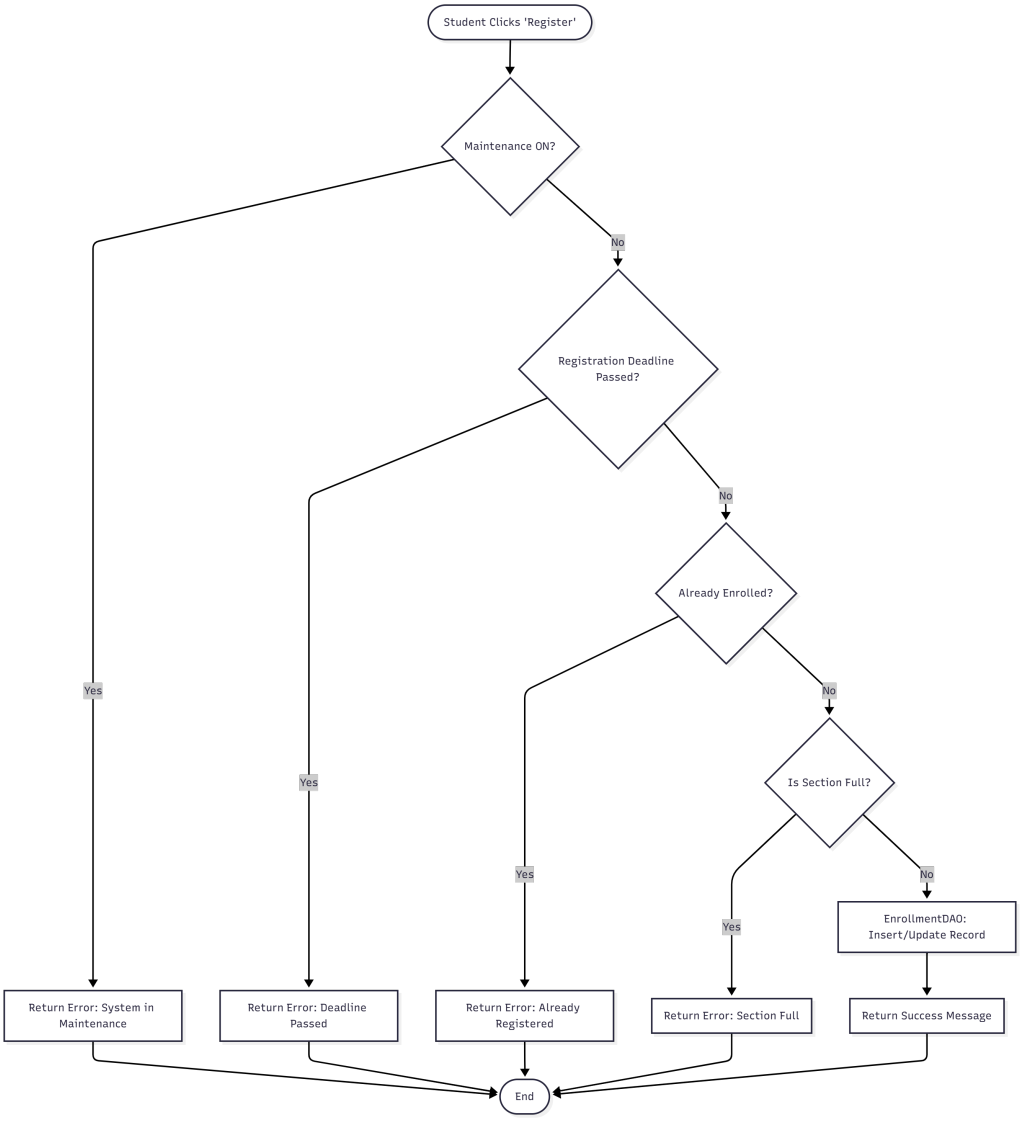


Figure 2: Student Enrollment Logic Flow

3.2 Grading and Calculation Flow

This flow illustrates the Instructor's gradebook process. It relies on the `AssessmentComponent` table to allow dynamic weighting schemas (e.g., changing a Final Exam weight from 50% to 60%) before computing the final letter grade.

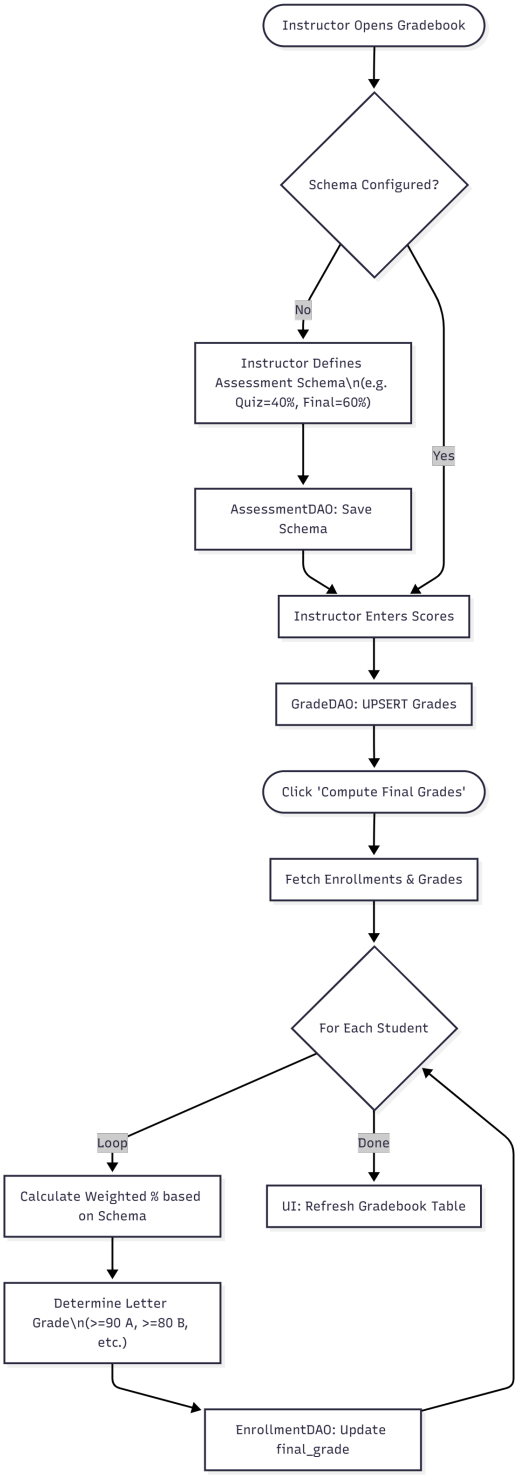


Figure 3: Grading and Final Grade Computation Flow

3.3 Maintenance Mode Toggle Flow

This flow shows how the Admin toggles system access. Instead of constant database polling, the system uses a **Callback** mechanism to update the UI immediately when the admin toggles the switch.

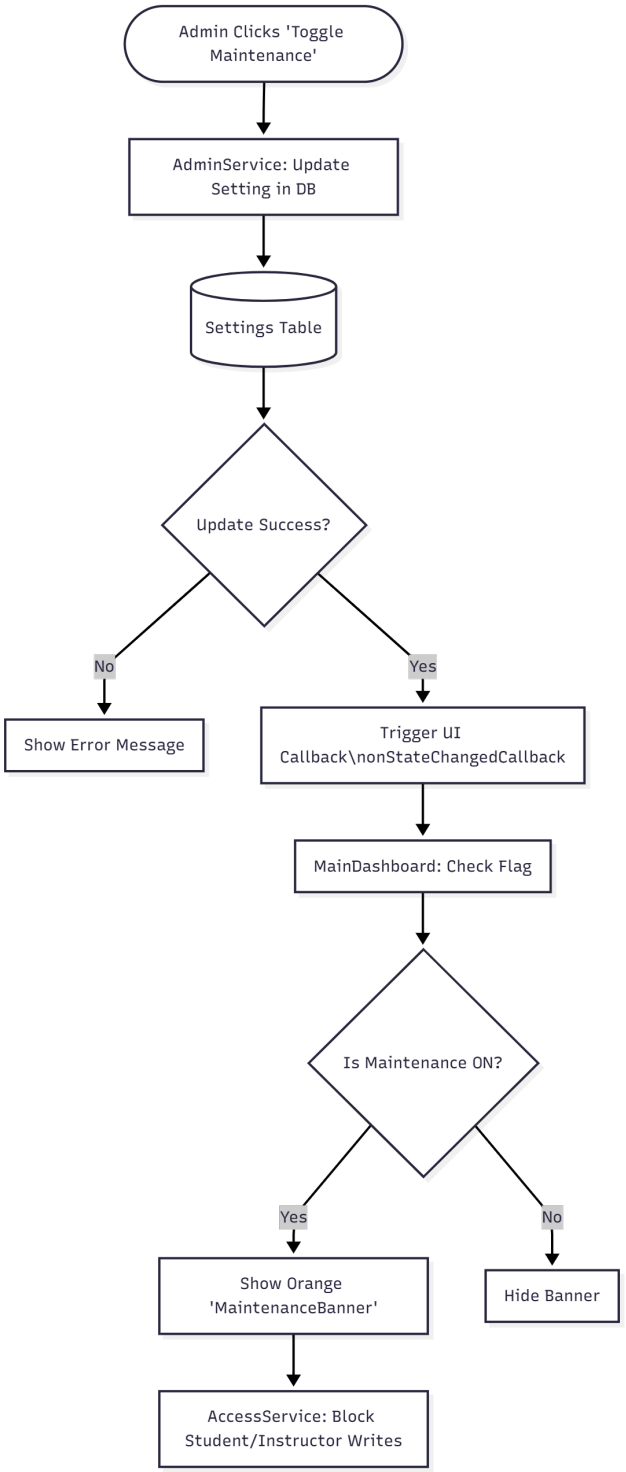


Figure 4: System Maintenance Toggle Flow